

Analysis of Apache Commons-Net Library

Master's Degree Course in Computer Science - IoT Curriculum
Software Dependability Project Report

apache/commons-net



Apache Commons Net

36

Contributors



53k

Used by



258

Stars



187

Forks



ABSTRACT

This report provides an analysis of **Apache Commons Net**, a Java library that offers a collection of network utilities for implementing various network protocols such as FTP, SMTP, POP3, and Telnet. The analysis focuses on the library's reliability, robustness, and maintainability in the context of software dependability.

1. INTRODUCTION

1.1 What is Apache Commons Net

Apache Commons Net is a widely used Java library that provides a comprehensive suite of utilities for implementing various network protocols, including FTP, SMTP, POP3, and Telnet. As part of the Apache Commons suite, it addresses the complexities of network communication by offering pre-implemented client APIs for net protocols.

It is widely adopted in both enterprise and open-source projects due to its flexibility and extensive support for industry-standard protocols. Furthermore, the library is continuously maintained and updated.

1.2 Project Evaluation Criteria

The following outlines the 9 project evaluation criteria, with a thorough analysis and corresponding outcomes for each category.

- **Buildability of the project** both in CI/CD and locally;

- **Software quality analysis with SonarCloud**, analyzing, categorizing and fixing issues;
- **Make a Docker image for the project**, ready to be orchestrated and being available in DockerHub;
- **Make the Docker container runnable**, within a web app or small Java application;
- **Code coverage analysis**, using Cobertura or Jacoco;
- **Mutation testing campaign**, to analyze the test cases of the project using PiTest;
- **Performance tests implemented using JMH**, to stress the most cumbersome project components;
- **Automated generation tools**, used to create tests that cover poorly tested code components;
- **Security analysis**, using OWASP FindSecBugs and OWASP DC.

2. PROJECT ANALYSIS

2.1 Information about environment and setup

A fork of the original repository was created to facilitate independent development and version control. Subsequently, the repository was cloned locally onto my computer using **GitHub Desktop**, allowing seamless synchronization between the remote

and local versions. **Visual Studio Code** was employed as the primary development environment. The system used for this setup was a Macbook Pro running **macOS Sequoia 15.0.1**.

2.2 Buildability of the project

The project has undergone thorough testing to ensure it is buildable both in a CI/CD pipeline and in local environments. All tests have been successfully executed, and the build process completes without any issues, confirming that the project is stable and fully compatible with both automated and local development workflows.

For the test's purposes, **Apache Maven** was used. Local builds used the **latest version of JDK 23**. As per CI/CD pipeline builds with GitHub, the latest version of Ubuntu running **JDK 23** was used.

GitHub project repository can be accessed [here](#).

2.3 Software quality analysis with SonarCloud

A thorough software quality analysis was conducted using SonarCloud, revealing several areas for improvement. The analysis identified **2 security issues**, **23 reliability issues**, and **1.3k maintainability issues**, **58 security hotspots** and a **duplication rate of 2.5%**.

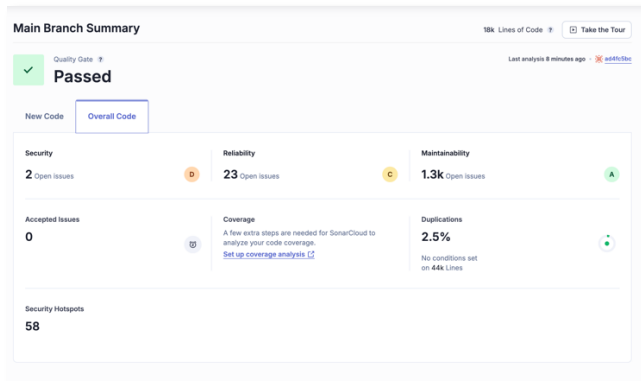


Figure 1. SonarCloud overall code analysis.

Most of the detected issues are categorized as medium and low severity, enabling a focus on the high-severity concerns in the following sections to ensure the most critical problems are addressed first, enhancing the overall security, reliability, and maintainability of the project.

2.3.1 Analysis and Categorization of Issues

After analyzing the project, the issues have been categorized as follows. A distinction will be made between **high**, **medium**, and **low** levels of severity of the issues.

Security Issues

Issues related to vulnerability, could be exploited by malicious users to perceive malicious intents. In total **2 security issues** were found:

HIGH	MEDIUM	LOW
2	0	0

Table 1. Security Issues overview.

Reliability Issues

Issues related to the stability and availability of the software, commonly referred to as bugs, could lead to errors or unexpected behavior in the code, and for this reason, they can be extremely dangerous. In total **23 reliability issues** were found:

HIGH	MEDIUM	LOW
0	21	2

Table 2. Reliability Issues overview.

Maintainability Issues

Issues related to code readability, structure, and the ability to implement changes without introducing new bugs or requiring extensive testing, commonly referred to as code smell, could lead to a difficult maintainability of the code. In total **1.3k maintainability issues** were found:

HIGH	MEDIUM	LOW
124	551	587

Table 3. Maintainability Issues overview.

Security Hotspots

Related to vulnerability, usually represent part of the code to analyze and review to determine whether it is a potential security issue or not . In total **58 security hotspots** were found:

HIGH	MEDIUM	LOW
0	1	57

Table 4. Security Hotspots overview.

3. ISSUES FIXING AND CODE REFACTORING

3.1 Security Issues

Two security issues were identified during the analysis, both categorized as "Enable server certificate validation on this SSL/TLS connection". The first issue was found in the method public void checkClientTrusted() in the class FTPSTrustManager. java, while the second issue concerns a same method but in the TrustManagerUtils. java class.

The solution to address both the identified security issues involve the implementation of a default TrustManager. This approach utilizes a TrustManagerFactory to create a TrustManager that loads Java's default TrustStore. The TrustStore contains the trusted certificates essential for both client and server validation.

In the checkClientTrusted() method, client validation is delegated to the default TrustManager, which checks whether the client certificates are valid and part of a known trust chain.

Similarly, the checkServerTrusted() method has been updated to delegate server validation to the default TrustManager, to maintain uniform code, even if not specifically highlighted by SonarCloud. This enhancement ensures a more thorough examination of the server certificate, complementing the previous manual validation performed by the already present checkValidity() method.

3.2 Maintainability Issues

A total of 1.3k issues related to code readability, structure, and the ability to implement changes without introducing new bugs or requiring extensive testing (commonly referred to as **code smell**) were identified during the SonarCloud analysis.

For the purposes of the project, only issues with high level severity were considered for fixing and refactoring, for a total of 124 issues, distributed as follows based on SonarCloud tags:

TAG	ACTION	# OF ISSUES
brain-overload	Refactor method to reduce its Cognitive Complexity	41
tests	Add at least one assertion to the test case	34
convention	Rename the constant name to match the regular expression	19
suspicious	Add a nested comment explaining why the method is empty, throw an UnsupportedOperationException or complete the implementation.	14
cwe	Add a default case to the switch	5
design	Define a constant instead of duplicating the literal	4
confusing	Rename field to prevent any misunderstanding	2
pitfall	Remove the non-escaped \u0007 character from the literal	2
serialization	Make * transient or serializable	2
multi-threading	Remove the call to "wait" or move it into a "while" loop	1

Table 5. Maintainability Issues in-depth categorization.

Following paragraphs will be named after the “TAG” column, to have a better visual perception of issues.

3.2.1 “Brain-overload” issues fixing

Brain-overload issues refer to blocks of code to refactor in order to reduce cognitive complexity. The highest value allowed for this parameter is 15. Every function below this number is considered to be valid. The majority of the issues found fall under this category.

The primary approach used was to refactor methods into smaller sub-methods to reduce cognitive complexity. The average complexity of the refactored methods ranges between 30 and 60, with some approaching 100 but which, due to their structure, were easily refactorable.

However, while most of the issues have been addressed, significant difficulties were encountered with only three of them.

Two of these methods, belonging to the `TFTPClient.java` class, have been partially refactored but still present cognitive complexities of 52 and 47, respectively.

The other issue, related to a method in the `TFTPClientExample.java` class, has a cognitive complexity of 159.

Therefore, since reducing their complexity would require, at the very least, a partial change in the software's logic along with a significant investment of time and resources, these three methods have been marked as “accepted issues” and left with a complexity above the acceptable threshold.

3.2.2 “Test” issues fixing

Test issues refer to the need for a validation step in unit tests to ensure that the code behaves as expected. In unit testing, an assertion is used to verify the outcome of the code being tested. Without an assertion, a test case doesn't actually verify any result, making it less useful because it doesn't check if the function or method produces the correct output or behavior.

34 issues were found during the analysis, 29 categorized as “Add at least one assertion to the test case” (hereinafter referred to as **Case A**) and 5 categorized as “Add a “`super. setUp()`” call to this method” (hereinafter referred to as **Case B**).

- **Case A:** 16 out of 29 issues were simply fixed adding at least one assertion to the test case without any trouble nor any logic or functionality change. The remaining 13 issues were natively implemented in other classes or empty auto-generated test cases, both ways labeled as “non needed/used” by comments, so since no implementation is present, no changes were made.
- **Case B:** no changes were made to the 5 issues due to “`super. setUp()`”. In fact, being an abstract method and not implemented, it was impossible to proceed with the correction without radically changing the logic of a large number of classes, potentially exceedingly complex and requiring significant time investment.

3.2.3 “Convention” issues fixing

Convention issues refer to code that does not comply to a predefined naming convention specified by a regular expression (regex).

19 issues were found during the analysis, all categorized as “Rename this constant name to match the regular expression ‘`^[A-Z][A-Z0-9]*(_[A-Z0-9]+)*$`’”, in nine different classes. An example of constants involved in the issue is “`__DEFAULT_ENCODING`”, clearly violating the naming convention.

Constants along with all references were correctly renamed to resolve issues.

3.2.4 “Suspicious” issues fixing

Suspicious issues refer to a method in the code that lacks any implementation. It can lead to confusion among developers, as it neither performs any actions nor communicates its intended purpose.

14 issues were found during the analysis, all categorized as “Add a nested comment explaining why this method is empty, throw an `UnsupportedOperationException` or complete the implementation”, in ten different classes. Methods analyzed did not have an implementation because, as indicated by some existing comments, they were no-ops by default, empty auto-generated test cases or implemented in other classes.

The issue was solved for all classes, adding more-explanatory comments where not present or needed.

3.2.5 “CWE” issues fixing

Common Weakness Enumeration (CWE) issues refer to a common recommendation in programming, e.g. for “add a default case to the switch” it refers to CWE-708: Missing Default Case in Switch Statement.

6 issues were found during the analysis, all categorized as “Add a default case to this switch”, in `FTPTimestampParserImpl.java`, `Telnet.java`, `TelnetInputStream.java`, `CompositeFTTParseTestFramework.java` and `TelnetClientTest.java` classes.

Default cases with break were added to every switch to solve the issue.

3.2.6 “Design” issues fixing

Design issues refer to a code quality concern where the same literal value (such as a number or string) appears multiple times throughout the codebase without being encapsulated in a constant variable.

4 issues were found during analysis, all categorized as “Define a constant instead of duplicating the literal”, in `FTPClientExample.java`, `IMAPMail.java`, `finger.java` and `IMAP.java` classes.

A constant for every class has been created, solving the issue.

3.2.7 “Confusing” issues fixing

Confusing issues refer to a naming conflict or ambiguity in the code that could lead to confusion or errors during development or maintenance. It is suggested that a variable or field name is not sufficiently descriptive or is too similar to another field, which may cause developers to misinterpret its purpose or functionality.

2 issues were found during the analysis, both categorized as “Rename field to prevent any misunderstanding/clash”, one in `TFTPDatapacket.java` class, the other in `TFTPErrorPacket.java` class.

Fields were simply renamed to prevent any misunderstanding and to enhance the readability of the code.

3.2.8 “Pitfall” issues fixing

Pitfall issues refer to a specific coding concern related to character encoding in programming languages regarding Unicode.

2 issues were found during the analysis, both categorized as “Remove the non-escaped `\u0007` character from this literal”, regarding two `System.err.println()` in the same class, `TFTPEXample.java`.

The characters were simply removed to solve the issue.

3.2.9 “Serialization” issues fixing

Serialization issues refer to the object serialization. This issue typically arises when a class contains non-serializable fields, but the class itself is intended to be serialized.

2 issues were found during the analysis, both categorized as “Make * transient or serializable”. The first one, `private final Object source`, was in `ProtocolCommandSupport.java` class, while the second one, `private final`

`CopyOnWriteArrayList<EventListener> listeners`, was in `ListenerList.java` class.

In both cases, declaration was changed to `transient`. The methods `readObject()` and `writeObject()`, found in both classes, remain unchanged, as they already throw an `UnsupportedOperationException`, reflecting that serialization is not supported for these classes.

3.2.10 “Multi-threading” issues fixing

Multi-threading issues refer to multithreading or concurrent programming.

Only 1 issue was found during the analysis, categorized as “Remove the call to “wait” or move it into a “while” loop”. The affected method was `final boolean _sendAYT()` in the `Telnet.java` class.

The `_sendAYT()` method was modified to incorporate the `wait()` call within a while loop. In this version, the method continues to check the `aytFlag` condition and remains in a waiting state until either the flag is set to true or the specified timeout elapses. To achieve this, a `remainingTime` variable is introduced, which calculates the time left for the wait based on the initial timeout and the elapsed time since the wait began.

3.3 Fixes and Refactoring Recap

In the following paragraph, a concise summary of the actions undertaken during the error-fixing and code refactoring process is provided.

The fixes and refactoring process encompassed a total of 126 issues, including 124 maintainability errors and 2 security-related vulnerabilities.

Of these, 123 issues were successfully addressed, resulting in a resolution rate of 98%. All 2 security issues were fully resolved, achieving a 100% resolution rate in this category. However, 3 general errors remain unresolved, representing 2% of the total issues.

CATEGORY	ISSUES	FIXED	UNFIXED
Maintainability	124	121 (98%)	3 (2%)
Security	2	2 (100.0%)	0 (0.0%)
Total	126	123 (98%)	3 (2%)

Table 6. Fixes overview.

Below, a snapshot of SonarCloud dashboard after the final analysis.

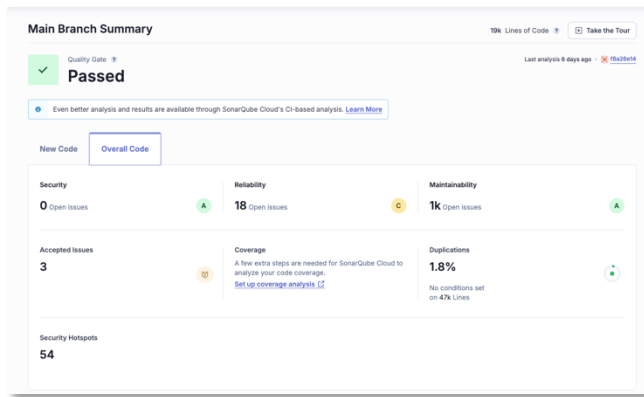


Figure 2. SonarCloud refactored code analysis.

4. DOCKER INTEGRATION

4.1 Main Goal

During this phase, a central role was played by Docker. A small web application capable of utilizing the modified library as a dependency was created. Subsequently, a Docker image was built and then run in a container.

The primary purpose of the web application was to verify the proper functionality of the library integrated as a dependency.

4.2 Web Application

The web application utilizes the Whois class from this commons-net library project, which implements the corresponding method that returns information regarding web domains.

Technologies such as Spring Boot and Thymeleaf were employed in the development of this application.

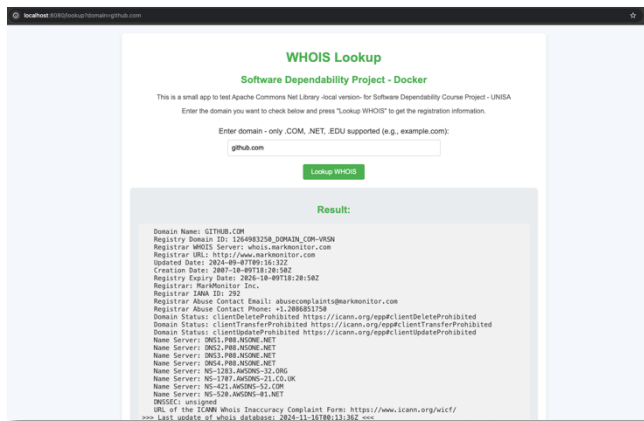


Figure 3. WHOIS web application implemented and running on Docker container.

The web application will work on localhost:8080. As Docker procedure, `docker run -d -p 8080:8080 <image-name>` command is needed to map 8080:8080 from container to local machine port.

Docker image of the web application can be found in Docker Hub repository [here](#).

5. TESTING

5.1 Code Coverage Analysis with JaCoCo: Insights and Results

Code coverage of the project is evaluated using JaCoCo, a widely used tool for analyzing the effectiveness of tests.

A report is generated, highlighting the percentage of code covered and identifying areas that are not adequately tested.

Below, a snapshot of JaCoCo dashboard after the analysis.

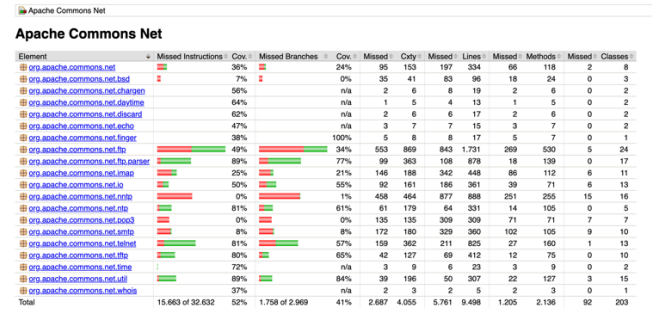


Figure 4. JaCoCo coverage report.

As can be observed, the code coverage is not significantly high, reaching an overall average of 52%.

It is also important to highlight several evident and critical issues:

- Uncovered portions of the code may potentially hide malfunctions or errors that remain undetected due to these limitations;
- POP3 and NNTP packages show a coverage rate of 0%.

5.2 Mutation Testing with PiTest

A mutation test was conducted on the project library using PiTest in its standard configuration, applying mutations classified under the "Stronger Group."

Below is a snapshot of the dashboard displaying the final results of the mutation test, along with an analysis of the data obtained from the process.

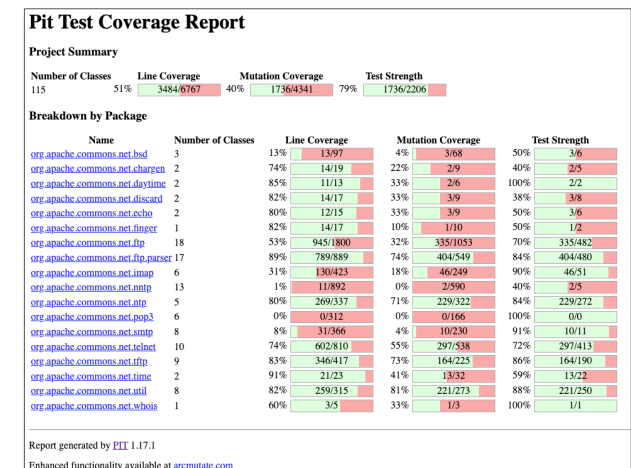


Figure 5. PiTest coverage report.

Data displayed on the dashboard, for mutated classes only, can be summarized as follows:

- Line Coverage is 51% (3484 lines on 6767);
- Mutation Coverage is 40% (killed 1736 mutations on 4341);
- Test Strength is 79%

Furthermore, additional in-depth details on each package and class are available in the report for a more thorough and meticulous analysis.

Packages that have overall achieved best results across the three parameters are:

Package	Line Coverage	Mutation Coverage	Test Strength
FTP.Parser	89%	74%	84%
Util	82%	81%	88%
NTP	80%	71%	84%

Table 7. Best overall packages by PiTest results.

While packages that have overall achieved worst results across the three parameters are:

Package	Line Coverage	Mutation Coverage	Test Strength
POP3	0%	0%	100% (0/0)
NNTP	1%	0%	40%
SMTP	8%	4%	91%

Table 8. Worst overall packages by PiTest results.

It is therefore evident that there is a need to review the effectiveness of the test suites, as well as to adapt them in order to extend coverage to the parts of the code that are currently not tested.

5.3 Improving the Coverage with auto-generated Test Cases

Given the low percentage of code coverage, automated test case generation tools were employed to achieve improved results. Although certain parts of the code exhibited significantly higher coverage percentages compared to others, it was decided to apply these tools across all packages of the project, as will be evident in the snapshots provided.

Finally, after employing all the tools, a summary table will be presented, illustrating how the coverage percentage has changed following the use of the various methodologies.

5.3.1 Randoop

Randoop successfully generated tests for 194 out of 219 classes, despite the Apache Commons-Net library classes not being particularly structured to accommodate this tool.

Ultimately, all pre-existing test cases, along with those created by Randoop, were executed, and the results are presented in the following snapshot by JaCoCo.

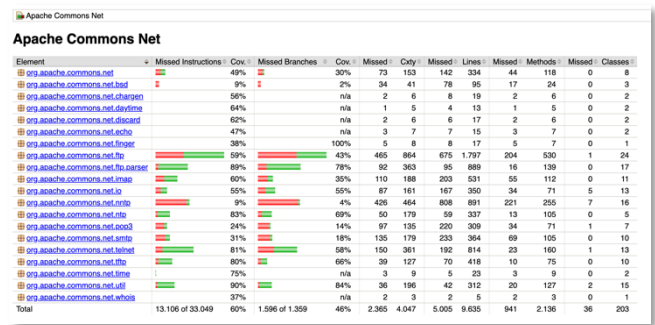


Figure 6. JaCoCo coverage report after Randoop.

As can be observed, there has been a moderate increase in coverage across nearly all packages, with particularly noteworthy improvements in some.

Even packages like POP3, which previously had 0% coverage, now achieve approximately 20%. Overall, the coverage percentage has risen from 52% to a total of 60%.

5.3.2 GitHub Copilot

Given the moderate and not particularly significant increase in code coverage percentage, GitHub Copilot was employed alongside Randoop to strive for better results.

Once again, test cases were either newly created or modified across all packages. The outcomes were analyzed using JaCoCo, as detailed in the following report. It is important to note that this final JaCoCo analysis encompasses all tests created thus far. In other words, the results represent a combination of pre-existing tests, those generated by Randoop, and those created or modified by Copilot.

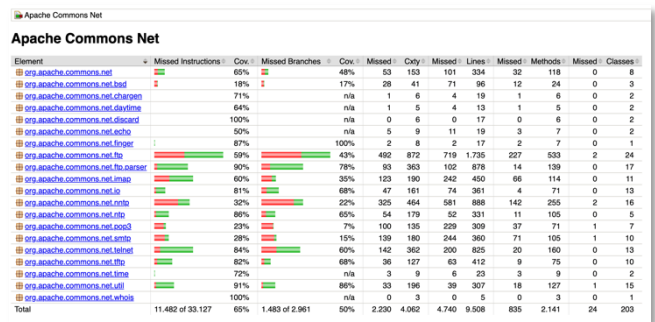


Figure 7. JaCoCo coverage report after Copilot.

As can be observed, there was a further increase in coverage across almost all packages. The overall percentage also improved by 5% compared to the previous analysis, bringing the code coverage to a total of 65%.

It can thus be concluded that the combined use of these tools yields significant benefits in terms of code coverage. Undoubtedly, even better and more professional results can be achieved by generating these tests collaboratively within a team and with a well-defined target in mind.

5.3.3 Final Results and Comparison

This paragraph presents the final results and the instructions coverage percentages obtained from the three JaCoCo analyses.

Lib.Tests: *Library Tests*; Tests natively in Commons-Net Library.

Package	JaCoCo Instructions Cov.		
	Built-in Tests		
		+Randoop	+Copilot
org.apache.commons.net	36%	49% ↑	65% ↑
org.apache.commons.net.bsd	7%	9% ↑	18% ↑
org.apache.commons.net.chargen	56%	56% ~	71% ↑
org.apache.commons.net.daytime	64%	64% ~	64% ~
org.apache.commons.net.discard	62%	62% ~	100% ↑
org.apache.commons.net.echo	47%	47% ~	50% ↑
org.apache.commons.net.finger	38%	38% ~	87% ↑
org.apache.commons.net.ftp	49%	59% ↑	59% ~
org.apache.commons.net.ftp.parser	89%	89% ~	90% ↑
org.apache.commons.net.imap	25%	60% ↑	60% ~
org.apache.commons.net.io	50%	55% ↑	81% ↑
org.apache.commons.net.nntp	0%	9% ↑	32% ↑
org.apache.commons.net.ntp	81%	83% ↑	86% ↑
org.apache.commons.net.pop3	0%	24% ↑	23% ↓
org.apache.commons.net.smtp	8%	31% ↑	28% ↓
org.apache.commons.net.telnet	81%	81% ~	84% ↑
org.apache.commons.net.tftp	80%	80% ~	82% ↑
org.apache.commons.net.time	72%	75% ↑	72% ↓
org.apache.commons.net.util	89%	90% ↑	91% ↑
org.apache.commons.net.whois	37%	37% ~	100% ↑
TOTAL	52%	60% ↑	65% ↑

Table 9. JaCoCo instruction coverage results comparison.

The improvement is clearly noticeable. While some packages show unchanged coverage between Randoop and Copilot, others experience a slight decrease of a few percentage points, possibly due to slight variations in percentages and approximation effects but also due to Copilot changes in test classes. For the purposes of this report, this data was considered valuable to retain and highlight, as it underscores potential gaps that may still exist in this category of emerging tools.

In this instance, it would be advisable to revert the changes made to the three classes that experienced a slight decrease in coverage, returning to the tests generated by Randoop.

However, there is still significant room for improvement. Modifying certain library classes might prove beneficial, enabling these tools to analyze the code more effectively and generate more robust test cases. Nevertheless, considering the initial starting point, the outcome can be regarded as commendable.

6. PERFORMANCE TESTING

6.1 Overview

The performance testing was conducted using JMH.

The class selected for analysis was `TelnetClient.java`, as it emerged during testing as one of the few with significantly longer testing durations compared to others. Consequently, it was decided to dive deeper into the analysis of this class, leading to the creation of `TelnetClientBenchmark.java` in accordance with JMH standards.

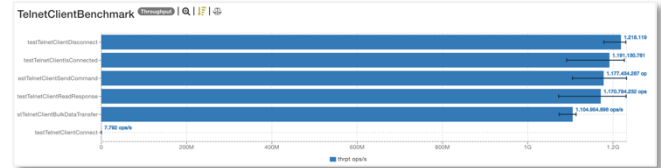


Figure 8. JMH results view through JMH Visualizer.

6.2 JMH Data Analysis

Results can be arranged as follows:

Operation	Throughput (ops/s)	Error (%)	Confidence 95%
Bulk Data Transfer	1,104,954,895 ops/s	~0.6%	[1,098M, 1,111M]
Connect	7,792 ops/s	~1.7%	[7,655, 7,930]
Disconnect	1,218,119,954 ops/s	~0.8%	[1,208M, 1,227M]
IsConnected	1,191,130,781 ops/s	~2.9%	[1,157M, 1,226M]
Read Response	1,170,784,231 ops/s	~3.8%	[1,126M, 1,215M]
Send Command	1,177,434,287 ops/s	~3.6%	[1,134M, 1,220M]

Table 10. JMH results.

6.2.1 Consistency and Error

The benchmarks demonstrate a good level of overall consistency. The Error values are relatively small compared to the scores, ranging from 0.6% to 3.8%. The use of 5 forks, each with 5 iterations for warmup and measurement, indicates that the results have been adequately stabilized.

6.2.2 Critical Operations

- Bulk Data Transfer: High throughput, though lower than operations like Disconnect or SendCommand. This is expected, as bulk data transfer demands more resources (CPU, I/O).
- Connect: Significantly slower than other operations, with throughput in the range of thousands of operations per second. This is typical for connections involving network handshakes.
- Disconnect, ReadResponse, SendCommand: These operations exhibit comparable performance, achieving throughput close to one billion operations per second. These results suggest that the code is highly optimized for these tasks.

7. SECURITY ANALYSIS

7.1 SonarCloud

During the analysis with SonarCloud, two security issues were identified. These issues have been addressed and resolved. A more detailed analysis, along with the resolution specifics, can be found in Chapter 3 - ISSUES FIXING AND CODE REFACTORING of this report, specifically in Section 3.1 - Security Issues.

7.2 FindSecBugs

An analysis was conducted using FindSecBugs, and an HTML report was generated. Specifically, the following issues were identified, as outlined in the screenshot below:

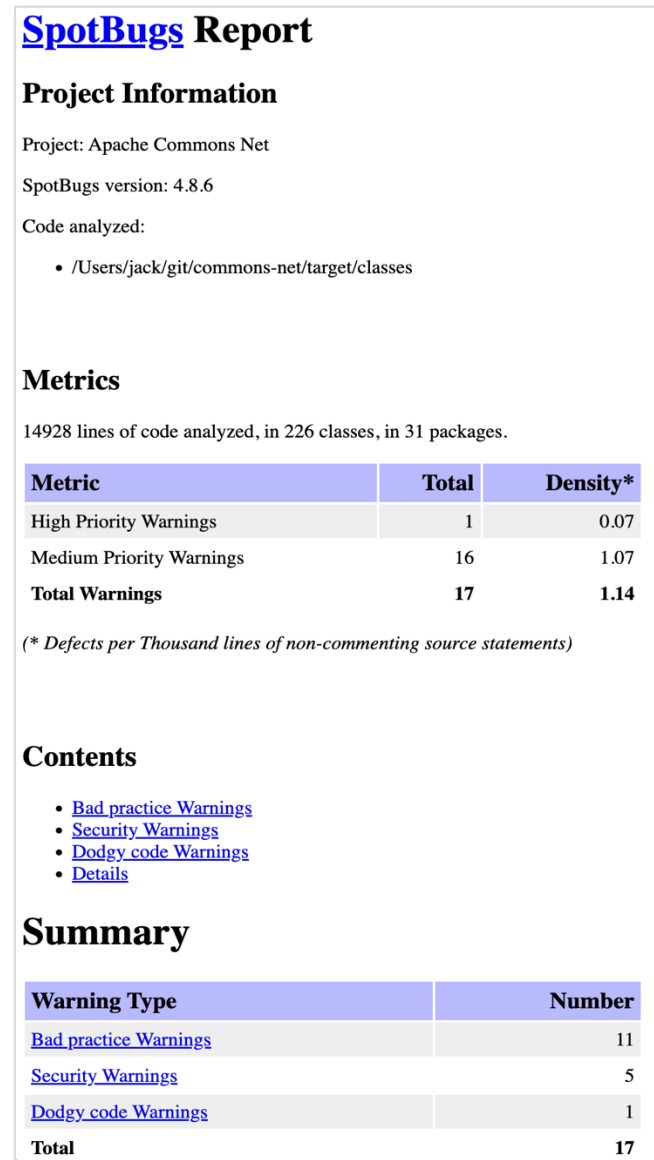


Figure 9. FindSecBugs report.

The identified security issues have been classified into the following categories:

- **Bad Practice Warnings:**

- Issues related to constructors leaving objects partially initialized, making them vulnerable to Finalizer attacks. These warnings were found in various classes, such as 'FTPTrustManager', 'RegexFTPFileEntryParserImpl', 'TimeInfo', 'TimeStamp', and 'SubnetUtils'.

- **Security Warnings:**

- *Predictable Randomness*: Use of 'java.util.Random', which is predictable and may weaken security.

- *ReDOS (Regular Expression Denial of Service)*: A vulnerable regular expression that could be exploited for denial-of-service attacks.

- *Unencrypted Communication*: Unencrypted sockets used in 'DelegateSocket' and 'FTPSCClient', which should be replaced with 'SSLSocket' for secure communication.

- *Weak TrustManager*: A TrustManager that accepts any certificates, exposing the application to Man-in-the-Middle attacks.

- **Dodgy Code Warnings:**

- *Dead Store*: An assignment to the variable 'name' in 'OS400FTPEntryParser.determineFileType()' that is not used, indicating unnecessary or incomplete code.

7.2.1 Bad Practice Warnings Fixing

Upon thorough review of the "Bad Practice Warnings", it can be confidently stated that all reported issues are false positives.

The warnings related to finalizer attacks are not applicable, as the affected classes do not use or rely on finalizers.

These classes adhere to modern Java practices, avoiding finalization in favor of explicit exception handling and resource management. When exceptions are thrown in the constructors, the objects are never left in an inconsistent state, as the constructors ensure full initialization or no object creation at all.

Therefore, the identified issues are not valid, and these warnings can be safely disregarded.

7.2.2 Security Warnings Fixing

Predictable Randomness: issue fixed using SecureRandom().

ReDOS: issue fixed simplifying the regular expression to avoid catastrophic backtracking.

Unencrypted Communication: the two issues were fixed using SSLSocket to enforce encryption.

Weak TrustManager: issue fixed modifying the method getAcceptedIssuers to return an array of accepted certificates instead of an empty array.

7.2.3 Dodgy Code Warnings Fixing

This issue was fixed removing the dead store to the local variable name in the determineFileType method.

7.3 OWASP DC

An analysis was conducted using OWASP Dependency-Check to assess vulnerabilities.

A total of 11 dependencies were examined, and no vulnerabilities of any kind were identified, as documented in the screenshot below.

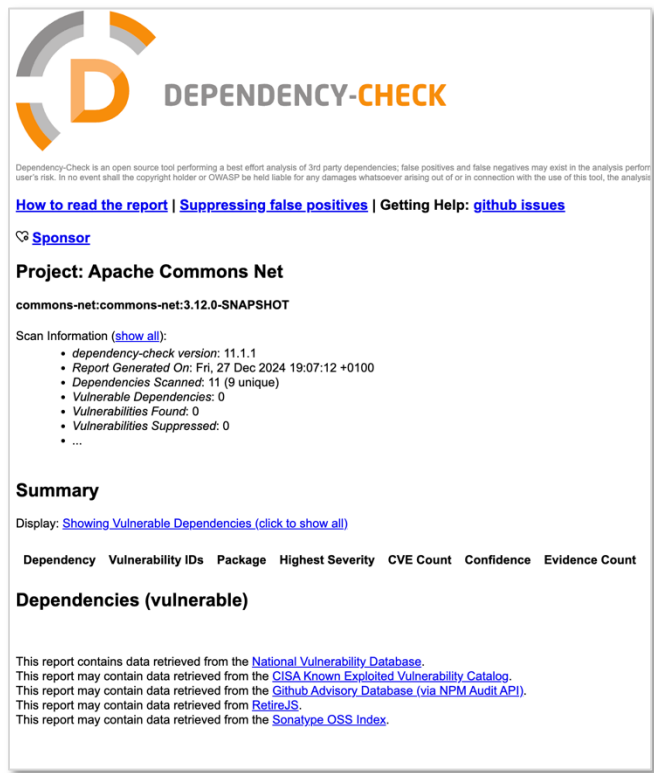


Figure 10. OWASP DC report.

8. CONCLUSIONS

In conclusion, the analysis of the selected library highlights the critical role of thorough evaluation and configuration in achieving software dependability.

Insights gained through this process contribute to enhanced reliability, compliance, and adaptability, supporting the development of trustworthy and robust software systems.